Chapter 1 Getting Started with JavaScript

CHAPTER OBJECTIVES

It looks like you've made the decision to dive into JavaScript — a great choice. JavaScript is a programming language capable of running on both the server side and the client side of an application. The server side refers to the backend logic, typically executed on machines housed in data centers that interact with databases, while the client side is whatever runs directly on the end user's device — most commonly inside a web browser, in JavaScript's case.

There's a good chance you've already encountered JavaScript in action without realizing it. If you've ever used a browser such as Chrome, Firefox, Safari, or Edge, you certainly have. JavaScript is everywhere on the web. When a site asks you to accept cookies and the popup vanishes after you click OK — that's JavaScript. When a sub-menu expands as you navigate a site — more JavaScript. Filtering products in an online store often relies on it too. And those chat widgets that pop up and start talking to you after you've lingered on a page for a few seconds? You guessed it — JavaScript again.

Nearly every interaction you have with a web page — the buttons you click, the birthday cards you design, the calculations you run — depends on JavaScript. Anything beyond a static page generally requires it.

This chapter covers the following topics:

- Why should you learn JavaScript?
- Setting up your environment
- How does the browser understand JavaScript?
- Using the browser console
- Adding JavaScript to a web page
- Writing JavaScript code

Why should you learn JavaScript?

There are many compelling reasons to learn JavaScript. The language dates back to 1995 and is widely regarded as the most extensively used programming language in the world, largely because it's the language that every web browser understands natively. If you have a browser and a text editor installed on your machine, you already have everything required to start running JavaScript — though, as we'll see later in this chapter, better setups do exist.

JavaScript is an excellent language for beginners, and even seasoned developers working in other languages will typically have some familiarity with it, simply because it's so hard to avoid. Several factors make it particularly well-suited to newcomers. For one, you can start

building genuinely impressive applications far sooner than you might expect. By the time you reach Chapter 5 on loops, you'll already be writing fairly sophisticated scripts that interact with users, and by the end of the book you'll be capable of building dynamic web pages that do all sorts of things.

JavaScript supports an enormous range of applications and script types. It can drive interactive behavior in the browser, but it can equally power the invisible backend logic of an application — including communication with databases — as well as games, automation scripts, and countless other use cases. It also accommodates multiple programming styles, or paradigms — different approaches to structuring and organizing code. Which style you adopt depends on what you're trying to build. If you've never written code before, these distinctions may not mean much yet, and that's fine — but it's worth knowing that JavaScript supports (semi-)object-oriented, functional, and procedural programming, among other paradigms.

Once you've got the fundamentals down, a vast ecosystem of libraries and frameworks becomes available to you. These tools dramatically extend what you can accomplish and how quickly you can do it. Well-known examples include React, Vue.js, jQuery, Angular, and Node.js. Don't worry about any of these for now — think of them as rewards waiting at the end of the journey. We'll touch on a few of them briefly near the close of this book.

Note: exercise, project, and self-check quiz answers can be found in the Appendix.

Finally, it's worth mentioning the JavaScript community itself. As one of the most widely used programming languages, JavaScript has an enormous base of practitioners — which means that as a beginner, you're unlikely to run into a problem that someone hasn't already solved and documented somewhere online. Stack Overflow, the popular developer Q&A forum, has an extensive section devoted entirely to JavaScript, and you'll likely find yourself there frequently while searching for solutions, tips, and tricks.

If JavaScript happens to be your very first programming language, you're stepping into the broader software development community for the first time — and you're in for a treat. Developers, regardless of which language they work in, tend to be generous about helping one another. Forums and tutorials abound, and answers to nearly any question are out there waiting to be found. As a newcomer, some of those answers may initially be hard to fully parse — that's completely normal. Stick with it, keep experimenting and learning, and the pieces will start falling into place before long.

Setting up your environment

There are many ways to configure a JavaScript development environment. In fact, your computer likely already has the bare minimum needed to start coding in JavaScript. That said, we recommend making your life easier by working in a proper IDE.

Integrated Development Environment

An Integrated Development Environment (IDE) is a specialized application for writing, running, and debugging code. You open it much like any other program — the way you'd open a word processor to write a document, select the right file, and start typing. Writing code follows a similar pattern: you open the IDE, write your code, and when you're ready to run it, most IDEs provide a dedicated button that executes the code directly from within the editor. For JavaScript specifically, you may sometimes find yourself opening a browser manually to see the result.

An IDE offers considerably more than basic file editing. Syntax highlighting is one of its most valuable features — different elements of your code are displayed in different colors, making it much easier to spot mistakes. Another valuable feature is autosuggest, often called code completion, where the editor proactively suggests valid options as you type. Many IDEs also support plugins that streamline integration with other tools — for example, enabling automatic browser refresh whenever your code changes (a "hot reload").

Many IDEs exist, each with its own strengths. This book uses Visual Studio Code throughout, though that's purely a matter of personal preference. Other popular options at the time of writing include Atom, Sublime Text, and WebStorm — though new IDEs appear regularly, so the most popular choice by the time you're reading this may well be different. A quick web search for "JavaScript IDEs" will turn up plenty of options. When choosing one, make sure it supports syntax highlighting, debugging, and code completion for JavaScript specifically.

Web browser

You'll also need a web browser. Most modern browsers work perfectly well for this purpose, though it's best to avoid Internet Explorer, which lacks support for the latest JavaScript features. Chrome and Firefox are both excellent choices — both support current JavaScript features and offer a wide range of helpful plugins.

Extra tools

There's a wide range of supplementary tools available — browser plugins, for instance, that assist with debugging or improve visual inspection of your code. None of these are essential at this stage, but it's worth staying open to trying out tools that others in the community recommend enthusiastically.

Online editor

If you don't have access to a full computer — perhaps just a tablet, or a laptop on which you can't install software — online code editors offer a great alternative. We won't recommend any specific ones by name, since this space evolves quickly and any recommendation here would likely be outdated by the time you're reading it. A quick search for "online JavaScript

IDE," however, will turn up plenty of viable options where you can write and execute JavaScript directly in your browser with the click of a button.

How does the browser understand JavaScript?

JavaScript is an interpreted language, meaning the computer processes and understands it on the fly, as it runs. Some languages require a separate processing step beforehand, called compiling — JavaScript does not; it can be interpreted directly. We'll refer to the component responsible for understanding JavaScript as the interpreter.

A web page is never composed of JavaScript alone — web pages are built from three distinct languages: HTML, CSS, and JavaScript.

HTML defines what content appears on the page. If a page contains a paragraph, that paragraph exists as HTML. A heading is likewise expressed in HTML. HTML is built from elements, also called tags, which specify the structural content of the page. Here's a minimal example that produces a page displaying the text "Hello world":

```
<html>
  <body>
    Hello world!
  </body>
</html>
```

CSS governs the visual layout of the page. Text color, font size, font family, and element positioning are all controlled through CSS.

JavaScript is the final piece of the puzzle — it defines what the page can actually do and how it interacts with the user or with the backend.

As you work with JavaScript, you'll inevitably encounter the term ECMAScript. This refers to the formal specification, or standardization, of the JavaScript language. The current standard is ECMAScript 6, commonly abbreviated ES6. Browsers implement JavaScript support based on this specification, alongside related specifications such as the Document Object Model (DOM), which we'll cover later. JavaScript has multiple implementations across different browsers, which can differ slightly in behavior, but ECMAScript represents the baseline specification that every implementation is expected to support.

Using the browser console

You may already be familiar with this, or perhaps not: web browsers include a built-in tool that exposes the underlying code powering the page you're viewing. Pressing F12 on a Windows machine while in your browser, or right-clicking and selecting "Inspect" on macOS, brings up a panel similar to the one shown in the accompanying screenshot. The exact

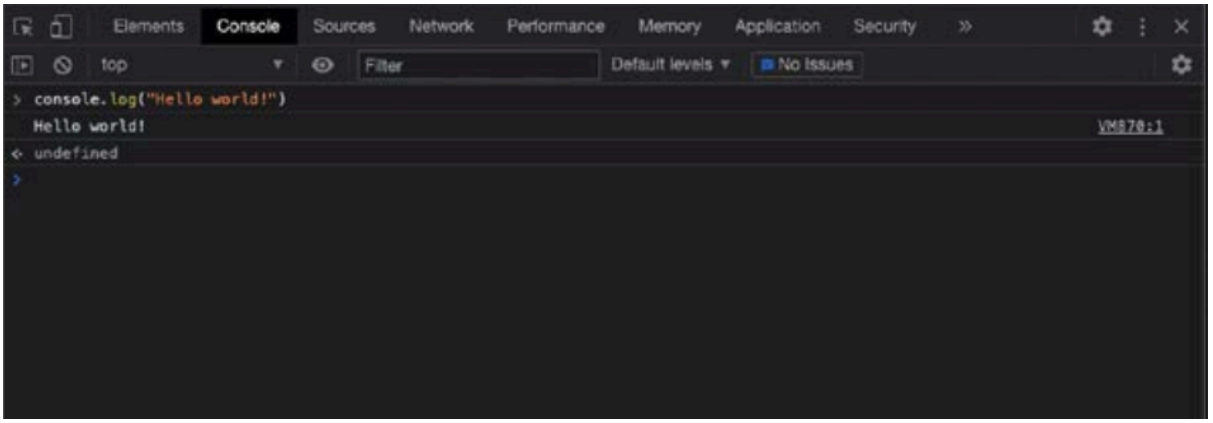
behavior may vary slightly depending on your browser and operating system, but right-clicking and choosing "Inspect" generally does the trick.

This panel contains several tabs along the top. The "Elements" tab, typically the default view, displays the page's HTML and CSS. Clicking the "Console" tab reveals a space at the bottom of the panel where you can type code directly. You may notice warnings or error messages appearing here — this is entirely normal and not necessarily a sign of anything wrong, even on functioning pages.

The console is primarily a tool developers use to log information about what's happening in their code and to perform debugging — the process of identifying the cause of unexpected or undesired behavior. Logging meaningful messages to the console provides useful insight into a script's execution. This brings us to our very first JavaScript command:

```
console.log("Hello world!");
```

If you open the console tab, type this line, and press Enter, you'll see the output displayed directly beneath it.



You'll be relying on `console.log()` extensively throughout this book to test code snippets and inspect results. Other useful console methods include `console.table()`, which formats array or object data as a visual table when appropriate, and `console.error()`, which logs a message with styling that draws attention to it as an error.

Practice exercise 1.1 — Working with the console:

1. Open the browser console, type `4 + 10`, and press Enter. What response do you see?
2. Use the `console.log()` syntax, placing a value inside the parentheses. Try entering your name surrounded by quotation marks (this denotes a text string — a concept we'll explore in the next chapter).

Adding JavaScript to a web page

There are two principal ways to link JavaScript to a web page. The first is to write JavaScript code directly within the HTML, enclosed between a pair of `<script>` tags. The opening `<script>` tag signals that executable script content follows; the content itself comes next; and the script is closed with a matching `</script>` tag, distinguished by the forward slash. Alternatively, a separate JavaScript file can be linked to the HTML document via a `<script>` tag, typically placed in the document's head.

Directly in HTML

Here's an example of a minimal web page that triggers a popup reading "Hi there!":

```
<html>
  <script type="text/javascript">
    alert("Hi there!");
  </script>
</html>
```

Saving this as an `.html` file and opening it in a browser produces the popup shown in the accompanying screenshot. We'll save this particular example as `Hi.html`.

The `alert` command generates a popup containing a message, with the message text specified inside the parentheses.

At this point, our content sits directly inside the `<html>` tags, which isn't best practice. A more proper structure introduces two child elements within `<html>`: `<head>` and `<body>`. The head element holds metadata and is also where external files are typically linked. The body holds the visible content of the page. We also need to declare the document type using `<!DOCTYPE html>`, since we're writing an HTML document. Here's the revised example:

```
<!DOCTYPE html>
<html>

<head>
  <title>This goes in the tab of your browser</title>
</head>

<body>
The content of the webpage
  <script>
    console.log("Hi there!");
  </script>
</body>

</html>
```

This page displays the text "The content of the webpage." If you check the browser console, you'll find a small surprise — the JavaScript has also executed, logging "Hi there!" to the console.

Practice exercise 1.2 — JavaScript in an HTML page:

1. Open your code editor and create an HTML file.
2. Within the file, set up the basic HTML structure: the doctype declaration, `<html>`, `<head>`, and `<body>` elements, followed by a `<script>` tag.
3. Place some JavaScript code inside the script tags — `console.log("hello world!")` works well.

Linking an external file to our web page

JavaScript can also be linked to an HTML document from a separate file — generally considered a better practice, since it keeps your codebase organized and prevents lengthy HTML files cluttered with embedded scripts. It also enables reuse: if the same JavaScript appears across ten pages of your site and needs to be modified, you only have to change a single file rather than updating every page individually.

First, create a separate JavaScript file, using the `.js` extension. We'll call ours `ch1_alert.js`, with the following content:

```
alert("Saying hi from a different file!");
```

Next, create a corresponding HTML file with this content:

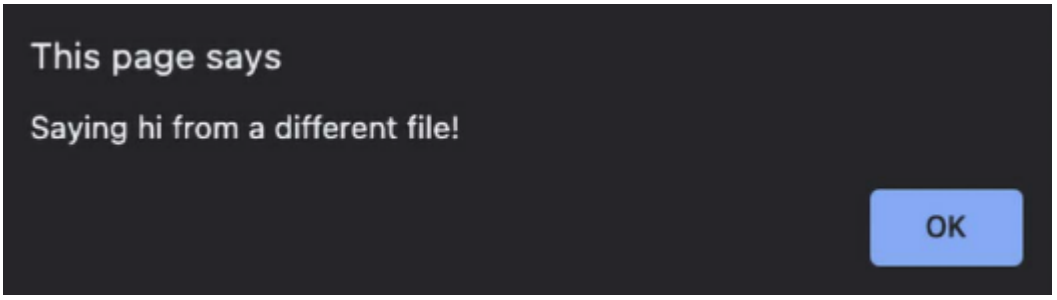
```
<html>
  <script type="text/javascript" src="ch1_alert.js"></script>
</html>
```

Make sure both files reside in the same location, or specify the correct path to the JavaScript file within the `src` attribute. File names are case-sensitive and must match exactly.

There are two approaches to specifying that path: relative and absolute. The absolute path is the simpler concept to explain first. Every computer has a root directory — `/` on Linux and macOS, and typically `C:/` on Windows. An absolute path describes the location of a file starting from that root. While easy to set up on your own machine, absolute paths have a significant drawback: if the website's files are later moved to a server, the absolute path will almost certainly break.

The safer alternative is the relative path, which describes how to reach the target file starting from the location of the current file. If the target file sits in the same folder, you need only its name. If it sits inside a subfolder called "example," you'd write `example/nameOfTheFile.js`. If it sits one folder up from the current location, you'd write `../nameOfTheFile.js`.

Opening the HTML file produces the popup shown in the accompanying screenshot.



Practice exercise 1.3 — Linking to a JS JavaScript file:

1. Create a separate file named `app` with a `.js` extension.
2. Add some JavaScript code inside the `.js` file.
3. Link that file to the HTML file you created in Practice exercise 1.2.
4. Open the HTML file in your browser and confirm that the JavaScript ran correctly.

Writing JavaScript code

With the necessary context now in place, let's turn to how JavaScript code is actually written. A few important habits to develop early on include proper code formatting, consistent indentation, correct use of semicolons, and meaningful comments. Let's start with formatting.

Formatting code

Well-formatted code is essential for readability. A long file containing many lines of code, written without adherence to a few basic formatting conventions, quickly becomes very difficult to follow. The two most important formatting practices to adopt right away are indentation and semicolon usage. Naming conventions matter too, and we'll address those as relevant topics arise throughout the book.

Indentations and whitespace

A given line of code frequently belongs to a particular code block — content enclosed between a pair of curly braces — or to a parent statement of some kind. When this is the case, the code within that block should be indented one level deeper, making it visually clear which lines belong to which block and where new blocks begin. You don't need to fully understand the example code below yet; it's included simply to illustrate the impact of indentation on readability.

Without any line breaks at all, the code becomes nearly unreadable. With line breaks but no indentation, it's somewhat more readable but still requires careful bracket-counting to determine where blocks begin and end. With both line breaks and proper indentation, the structure becomes immediately apparent — you can see at a glance which closing brace corresponds to which opening statement, without having to count brackets manually. Even though indentation has no effect on whether code actually runs correctly, developing the habit of using it consistently will pay off enormously as your programs grow more complex — you'll thank your past self later.

Without new lines:

```
let status = "new"; let scared = true; if (status === "new") { console.
log("Welcome to JavaScript!");  if (scared) { console.log("Don't worry
you will be fine!"); } else { console.log("You're brave! You are going
to do great!"); } } else { console.log("Welcome back, I knew you'd like
it!"); }
```

With new lines but without indentation:

```
let status = "new";
let scared = true;
if (status === "new") {
console.log("Welcome to JavaScript!");
if (scared) {
console.log("Don't worry you will be fine!");
} else {
console.log("You're brave! You are going to do great!");
}
} else {
console.log("Welcome back, I knew you'd like it!");
}
```

With new lines and indentation:

```
let status = "new";
let scared = true;
if (status === "new") {
  console.log("Welcome to JavaScript!");
  if (scared) {
    console.log("Don't worry you will be fine!");
  } else {
    console.log("You're brave! You are going to do great!");
  }
} else {
  console.log("Welcome back, I knew you'd like it!");
}
```

Semicolons

Every statement should end with a semicolon. JavaScript is fairly forgiving and will often correctly interpret code even when a semicolon has been omitted, but it's a good habit to develop early regardless. Code blocks themselves — such as the body of an `if` statement or a loop — should not be terminated with a semicolon; semicolons apply only to individual statements.

Code comments

Comments instruct the interpreter to skip over certain lines entirely — commented lines are never executed. This proves useful for several reasons:

1. Temporarily disabling a piece of code without deleting it, so it's ignored during execution.
2. Recording metadata — for instance, noting the author of a file or summarizing what it does.
3. Explaining specific sections of code — clarifying what a block does or why a particular approach was chosen.

JavaScript supports two comment styles: single-line and multi-line. Single-line comments begin with `//`, and everything following that marker on the same line is ignored — this can also be used as an inline comment appended to the end of a line of code. Multi-line comments begin with `/*` and end with `*/`, with everything in between ignored, regardless of how many lines it spans.

```
// I'm a single line comment
// console.log("single line comment, not logged");

/* I'm a multi-line comment. Whatever is between the slash asterisk and
the asterisk slash will not get executed.
console.log("I'm not logged, because I'm a comment");
*/
```

Practice exercise 1.4 — Adding comments:

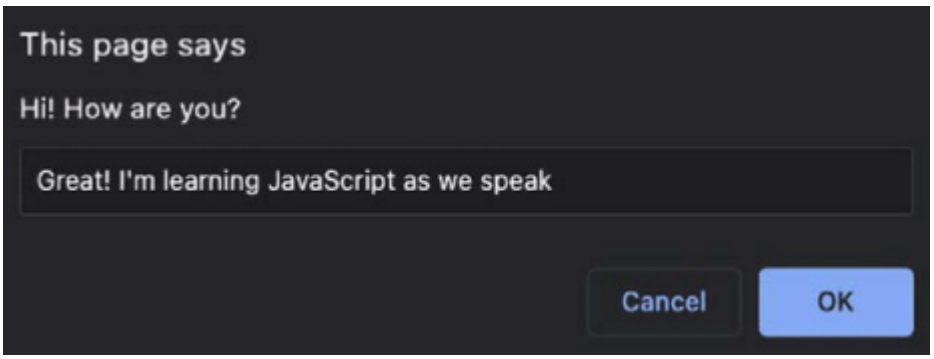
1. Add a new statement to your JavaScript code that sets a variable's value. Since variables are covered in the next chapter, you can simply use: `let a = 10;`
2. Add a comment at the end of that line indicating that you've set the value to 10.
3. Print the value using `console.log()`, along with a comment explaining what this line does.
4. At the end of your script, add a multi-line comment. In a real production script, this space might be used to provide a brief summary of the file's purpose.

Prompt

Another useful tool worth introducing here is the command prompt. It behaves much like `alert`, except that it accepts input from the user rather than just displaying a message. We'll cover how to store values in variables shortly, and once you understand that, you'll be able to capture the result of a prompt and use it within your code. Try modifying the `alert()` call in your `Hi.html` file to a `prompt()` call instead, like this:

```
prompt("Hi! How are you?");
```

Refresh the page, and you'll see a popup containing a text input box where you can type a response. Whatever value you (or any other user) enter is returned to the script and becomes available for use in your code — a powerful mechanism for gathering user input that shapes how your program behaves.



Random numbers

For the playful exercises in the early chapters of this book, it's useful to know how to generate a random number in JavaScript. It's perfectly fine if the underlying mechanics aren't fully clear yet — for now, just know that this is the command that generates a random number:

```
Math.random();
```

You can run this directly in the console and observe the result by logging it:

```
console.log(Math.random());
```

This produces a decimal value between 0 and 1. To scale it to a number between 0 and 100, multiply by 100:

```
console.log(Math.random() * 100);
```

If a whole number is preferred rather than a decimal, the `Math.floor` function rounds the result down to the nearest integer:

```
console.log(Math.floor(Math.random() * 100));
```

Don't worry if this doesn't fully click yet — it will be explained in much greater depth later in the book. Chapter 8, "Built-In JavaScript Methods," covers built-in methods like these in detail. Until then, simply trust that this expression reliably produces a random whole number between 0 and 100.

Summary

Nicely done — you've made your first real start with JavaScript! In this chapter, we covered a substantial amount of foundational context that you'll need before diving into actual coding. We saw that JavaScript serves many purposes, with the web being among its most prominent use cases. Browsers are able to run JavaScript because they include a dedicated component — the interpreter — capable of processing it directly.

We explored several options for setting up a JavaScript development environment on your computer, settling on the recommendation to use an IDE — a dedicated program for writing and running code. We also covered multiple ways to incorporate JavaScript into a web page: embedding it directly within `<script>` tags, and linking to a separate external JavaScript file.

We closed the chapter with some essential guidance on writing well-structured, readable, and maintainable code, supported by clear and purposeful comments. Along the way, we learned how to write output to the console using `console.log()`, how to gather user input with `prompt()`, and how to generate random numbers using `Math.random()`.

Next, we'll turn our attention to JavaScript's basic data types and the operators used to manipulate them.